that is an instance of a class that has included or prepended the module somewhere in it's ancestor list.

Outside a class or module definition, a definition with an unadorned method name is added as a private method to class Object. It may be called in any context without an explicit receiver.

A definition using a method name of the form *expr.methodname* creates a method associated with the object that is the value of the expression; the method will be callable only by supplying the object referenced by the expression as a receiver. This style of definition creates per-object or *singleton methods*. You'll find it most often inside class or module definitions, where the *expr* is either self or the name of the class/module. This effectively creates a class or module method (as opposed to an instance method).

```
class MyClass
  def MyClass.method      # definition
  end
end

MyClass.method            # call

obj = Object.new

def obj.method            # definition
end

obj.method                # call

def (1.class).fred        # receiver may be an expression
end

Integer.fred               # call
```

Method definitions may not contain class or module definitions. They may contain nested instance or singleton method definitions. The internal method is defined when the enclosing method is executed. The internal method does not act as a closure in the context of the nested method—it is self-contained.

```
def toggle
  def toggle
    "subsequent times"
  end
  "first time"
end

toggle    # => "first time"
toggle    # => "subsequent times"
toggle    # => "subsequent times"
```

The body of a method acts as if it were a begin/end block, in that it may contain exception-handling statements (rescue, else, and ensure).

Method Arguments

A method definition may list zero or more positional arguments, zero or more keyword arguments, a optional splat argument, an optional double splat argument, and an optional block argument. Arguments are separated by commas, and the argument list may be (and